Network Intrusion Detection System Powered by Machine Learning

Phase 1: Analysis and Design

Amira BOUAFIF & Hajer JEMAA

2025-2026

Introduction

With the increasing number of cyberattacks targeting modern networks, detecting malicious activity has become an essential task for organizations. Traditional security systems often struggle to keep up with the volume and complexity of current network traffic. For this reason, machine learning techniques are increasingly used to improve intrusion detection.

This project focuses on the design and analysis of a Network Intrusion Detection System (NIDS). In this phase, we define the technical architecture, select the dataset, choose appropriate algorithms and the evaluation metrics. We also present the use case, class and architecture diagrams.

1 Technical Stack

- **AI Engine:** Python (TensorFlow, PyTorch, scikit-learn, XGBoost).
- **AI microservice:** FastAPI (lightweight, high-performance API framework).
- **Backend:** Spring Boot (robust REST APIs, scalable, enterprise-ready).
- **Database:** MySQL.
- **Frontend:** React (custom dashboard for monitoring and analytics).
- **Authentication:** Spring Security + JWT.
- **Real-time:** Server-Sent Events (SSE).
- **Model Serialization:** Joblib (.pkl).
- **Deployment:** Docker containers; optional Kubernetes for scalability.

2 Dataset Choice

Property	**Value**
Dataset	CIC-IDS2017
Source	Canadian Institute for Cybersecurity
Format	CSV (pre-extracted features)
Size	$\approx$ 2.8M records across 8 CSV files
Feature count	78 features per flow record
Attack types	DoS, DDos, Probe, Brute Force, Web Attack, Botnet, Infiltration

Table 1: Dataset properties

Justification

- Comprehensive labeled dataset with multiple attack types (DoS, DDoS, Brute Force, etc.)
- Widely used benchmark, realistic traffic, suitable for ML/DL experiments.

3 Algorithms Choice

- **Machine Learning:** Decision Tree (baseline), XGBoost (main model).
- **Anomaly Detection (future work):** Autoencoders, hybrid chained/parallel approaches.

Justification for the XGBoost model:

- High accuracy on CIC-IDS2017 benchmark
- Fast inference time suitable for near real-time detection
- Low memory footprint for API deployment via FastAPI
- No GPU requirement as it is deployable on standard hardware

4 Metrics

Metric	**Target**	**Priority**	**Justification**
Accuracy	$\geq 98\%$	High	Standard benchmark on CIC-IDS2017
F1-Score (macro)	≥ 0.95	High	Handles class imbalance across attack types
False Positive Rate	$\leq 2\%$	Critical	Too many false alerts causes alert fatigue
False Negative Rate	$\leq 1\%$	Critical	Missed attacks represent a security breach
Inference Time	$\leq 200ms$	High	Near real-time requirement

Table 2: Target Metrics

5 AI / Detection Engine

- Data preprocessing pipeline (cleaning, normalization, encoding)
- XGBoost training and evaluation
- Model serialization (.pkl file via Joblib)
- FastAPI microservice exposing `POST /predict` endpoint
- Input: CIC-IDS2017 feature vector (78 features)
- Output: { `label: String, confidence: float` }

6 Backend

- REST API for alert management (CRUD)
- REST API for network flow submission and storage
- Integration with FastAPI via HTTP client (WebClient)
- Alert, NetworkFlow and AuditLog persistence in MySQL
- Auto-generation of weekly reports
- Server-Sent Events (SSE) endpoint for real-time alert push
- JWT authentication (login / logout)
- Role-based access control (`ADMIN` / `IT_MANAGER`)

7 Frontend

- Login page
- Dashboard : real-time alert counter, attack type distribution chart, network flow volume over time
- Statistics Summary page : aggregated security metrics
- Alerts List page : paginated and searchable
- Alert Detail page : full flow data and status management (`REVIEWED` / `DISMISSED`)
- Reports page : view auto-generated reports and export
- Real-time updates via Server-Sent Events (SSE)

8 Modeling

8.1 Use Case Diagram

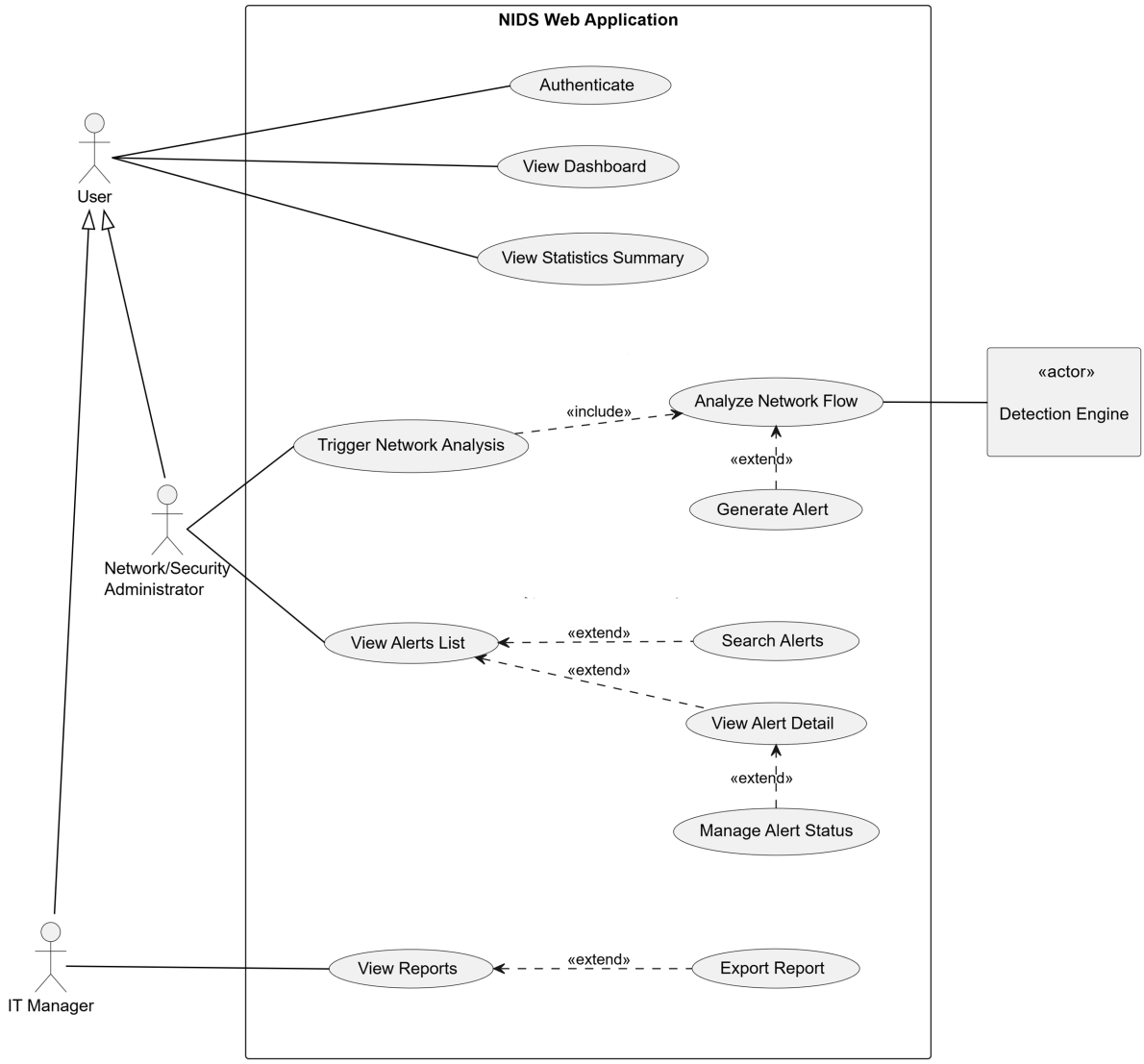

Figure 1: Use Case Diagram

8.2 Class Diagram

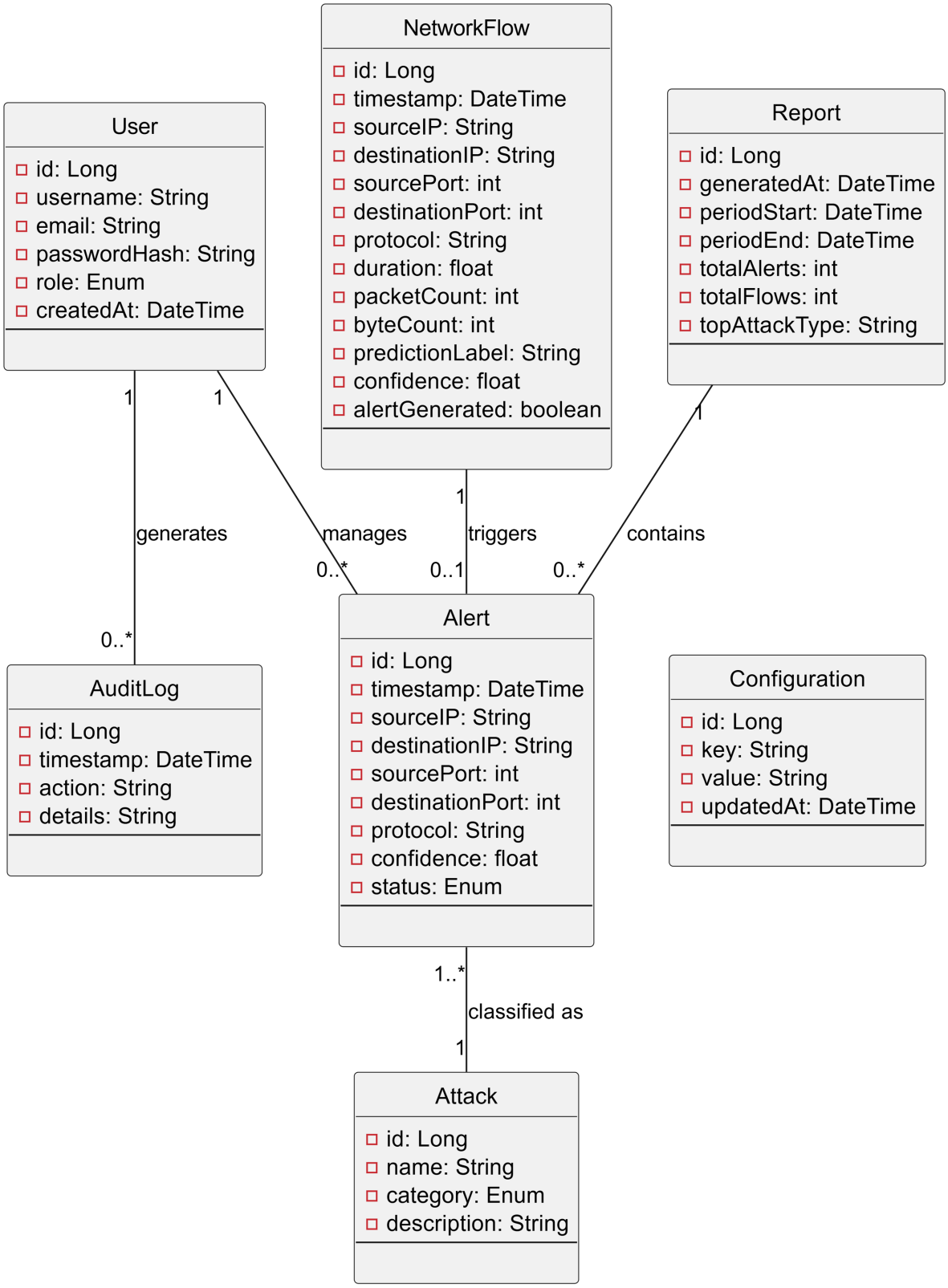

Figure 2: Class Diagram

8.3 Architecture Diagram

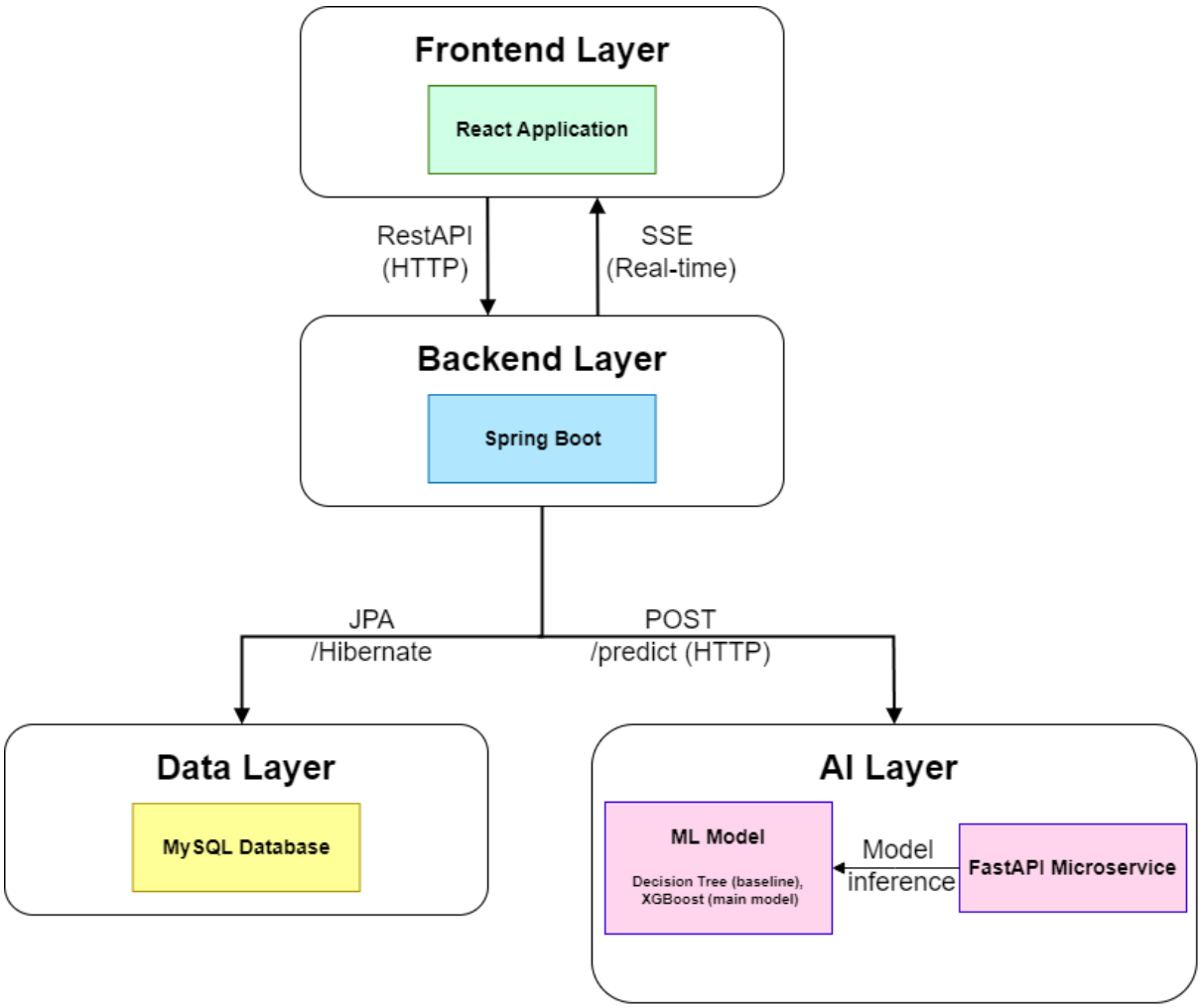

Figure 3: Architecture Diagram

9 Product Backlog

Weeks	Tasks	Deliverables
1–5	Data understanding + preparation	Preprocessing scripts, dataset report
6–10	Baseline ML models (Decision Tree, XGBoost)	Trained models, evaluation metrics
11–12	Backend + Frontend integration	Spring Boot APIs, React dashboard
13–14	Testing + reporting	Validation, documentation, and reporting End-to-end demo, detailed logs, visualizations, automated security report, final presentation

Conclusion

This phase established the foundation of the project by defining the system architecture, selecting the dataset, and identifying the machine learning models that will be used for intrusion detection. The proposed solution combines a machine learning detection engine with a web-based monitoring platform, allowing security alerts to be analyzed in real time. The next stages of the project will focus on the data understanding and preprocessing.